

Dataset A: A Stratified Multi-Capability Benchmark for Routing Evaluation of Large Language Models

Massa, F.

francescomassa06@gmail.com

May 2026

Abstract

We present **Dataset A**, a stratified evaluation benchmark of **5,339 queries** spanning six capability dimensions (instruction following, coding, mathematical reasoning, world knowledge, creative synthesis, and agentic planning). The dataset is designed to evaluate three large language models (**qwen3.5-9b**, **deepseek-v4-flash**, **kimi2.6**) and, in subsequent phases, three routing systems (FrugalGPT-style, RouteLLM-style, and a custom semantic router). Each row carries a deterministic identifier, a typed evaluation payload, pre-computed token counts under three tokenizers, and a license/gated tag enabling compliant public redistribution. We describe the construction pipeline (download, normalization, few-shot composition, tokenization, stratification, push), the iterative quality-validation procedure (deterministic checks, LLM-as-judge, dual-reviewer manual audit), and the remediation cycles that produced the current revision (**v0.3.0**, HF Hub revision **73aa0a09**). A dual-reviewer manual audit on $n = 102$ stratified samples yields Cohen’s $\kappa = 0.761$ (4-class rubric) and $\kappa = 0.951$ (binary), exceeding the standard substantial-agreement threshold of 0.70. As an initial application, we report a single-model capability evaluation of all three target models (**qwen3.5-9b**, **deepseek-v4-flash**, **kimi2.6**) on all ten scoring protocols, with the agentic planning rubric additionally evaluated under a three-judge panel to characterize judge-induced variance on the most subjective protocol. The dataset is publicly available on the Hugging Face Hub at <https://huggingface.co/datasets/massaindustries/dataset-A-routing-eval>.

1 Introduction

Routing systems for large language models (LLMs) require evaluation benchmarks that (i) exercise a broad set of capabilities, (ii) admit deterministic per-row scoring, and (iii) include the ancillary metadata required to derive per-model success and per-call cost. Existing single-capability benchmarks (MATH [2], GSM8K [3], MMLU-Pro [4], IFEval [5], BFCL [7], LiveCodeBench [8], SimpleQA [9]) are well-suited to scoring individual models, but no single corpus provides a stratified, multi-capability, license-clean substrate suitable for routing-level decision evaluation.

Dataset A fills this gap. It is stratified over six dimensions, sourced from fourteen upstream corpora, normalized to a fourteen-column schema with a discriminated-union evaluation payload, and tokenized under the three target models’ tokenizers (one exact, two proxy-grade). The dataset was iteratively constructed and validated across three revisions; the final revision (**v0.3.0**) passes all production-readiness gates including a dual-reviewer manual audit.

The remainder of this document describes the composition (§2), schema (§3), construction pipeline (§4), few-shot strategy (§5), tokenization (§6), gated masking (§7), the three-tier quality validation (§8), the iterative remediation cycles (§9), the evaluation results (§10), limitations (§11), and distribution (§12).

2 Composition

Dataset A is partitioned across six capability dimensions, each populated from one or more upstream sources. Table 1 reports per-source counts, licenses, and the few-shot policy applied at assembly time.

Table 1: Per-source composition of Dataset A (v0.3.0, $N=5,339$).

Dimension	Source	Count	License	Shots
instruction_following	IFEval [5]	541	apache-2.0	5
	IFBench [6]	300	ODC-BY-1.0	5
coding	LiveCodeBench-v6 [8]	1,000	cc-by-4.0	5
math_reasoning	MATH-500 [2]	500	mit	5
	AIME-2025	30	qwen-derived	5
	GSM8K [3]	470	mit	5
world_knowledge	SimpleQA [9]	700	mit	0
	MMLU-Pro-Humanities [4]	102	mit	5
	GPQA-Diamond [10]	0 (gated)	cc-by-4.0	5
creative_synthesis	EQ-Bench-Creative-v3 [12]	96	EQ-Bench terms	5
	LitBench [13]	500	qwen-derived	5
	Custom-Validated	100	qwen-derived	0
planning_agentic	BFCL-v4 [7]	500	apache-2.0	0
	tau-bench [14]	165	mit	0
	Planning-Custom	335	qwen-derived	0
Total		5,339		

Three dimensions reach the nominal target of 1,000 rows (*coding*, *math_reasoning*, *planning_agentic*). Three dimensions are below target by deliberate design choices documented in §11: *instruction_following* ($n=841$, with the upstream IFEval test split capped at 541 rows), *world_knowledge* ($n=802$ before gated masking; GPQA-Diamond is masked in the public release), and *creative_synthesis* ($n=696$, with the Custom-Validated subset capped at 100 rows pending incremental human-in-the-loop expansion).

3 Schema

Each row is a JSON object conforming to the 14-column schema below. The `expected_answer` field is a discriminated union over an enumeration of payload types.

Listing 1: Example record from Dataset A.

```
{
  "query_id": "q_00147",
  "query": "<full prompt including 5-shot CoT prefix>",
  "dimension": "coding",
  "source": "LiveCodeBench-v6",
  "shots": 5,
  "input_tokens_qwen": 1825,
  "input_tokens_deepseek": 1742,
  "input_tokens_kimi": 1768,
  "expected_answer": {
    "type": "test_cases",
    "payload": {"public_tests": [...], "private_tests": "<base64 zlib>"}
  },
}
```

```

"few_shot_examples": [
  {"question": "...", "reasoning": "...", "final_answer": "..."},
  ... (5 entries)
],
"evaluation_protocol_id": "lcb_unit_test",
"gated": false,
"license": "cc-by-4.0",
"length_band": "long"
}

```

The eight admissible values for `expected_answer.type` are: `test_cases` (LiveCodeBench public/private I/O pairs), `exact_match` (GSM8K final-answer string), `math_equivalent` (MATH-500, AIME-2025; sympy-equivalence judging), `tool_call_match` (BFCL ground-truth function-call dictionaries), `tool_call_trajectory` (tau-bench multi-turn trajectories), `rubric_judge` (creative + custom-validated; LLM-as-judge with explicit rubric), `ifeval_constraint` (IFEval/IFBench instruction-checker kwargs), `mcq_letter` (MMLU-Pro single-letter answer), `llm_judge_factual` (SimpleQA semantic judging).

4 Construction Pipeline

The pipeline is split into thirteen idempotent stages, each backed by a single Python script under `scripts/`. All stages are seeded with a deterministic constant (`seed=42`); per-source upstream revisions and SHA-256 hashes of derived artifacts are pinned in `data/reports/lockfile.yaml`.

1. **Setup check** (`00_setup_check.py`). Validates Hugging Face authentication, gated dataset access tokens, and resolves tokenizer identifiers for the three target models.
2. **Per-source download** (`10_download_<source>.py`, fourteen scripts). Pulls each upstream dataset from the Hugging Face Hub, persisting raw rows to `data/raw/<source>.jsonl` together with the upstream commit hash. BFCL is fetched via `huggingface_hub.list_repo_files` (the dataset script is incompatible with `load_dataset`); the question file and the corresponding `possible_answer/` file are joined on row id to populate `ground_truth_calls`.
3. **tau-bench materialization** (`11_clone_taubench.py`). Clones `sierra-research/tau-bench`, installs editable, imports `tau_bench.envs.{retail,airline}.tasks`, and serializes to JSONL.
4. **EQ-Bench fetch** (`12_fetch_eqbench.py`). HTTP GET against the `eqbench.com` manifest endpoint; 96 prompts persisted with the canonical creative-rubric envelope.
5. **Per-dimension normalization** (`20_normalize_<dimension>.py`, six scripts). Maps native fields to the target schema; SimpleQA receives an explicit format-prefix instruction; BFCL ground-truth-calls validation enforces non-empty payload for non-irrelevance categories.
6. **Native-pool few-shot extraction** (`30_fewshot_extract.py`). For sources with a native train split (GSM8K, MMLU-Pro, LitBench, IFEval), five exemplars are sampled and persisted disjoint from the evaluation queries.
7. **Synthetic few-shot generation** (`30b_fewshot_regolo_generate.py`). For sources without a native train pool (MATH-500, AIME-2025, LiveCodeBench-v6, IFBench, GPQA-Diamond, EQ-Bench-Creative-v3), five exemplars are generated by `qwen3.5-122b@regolo` using a source-specific meta-prompt with explicit format constraints (e.g. for LiveCodeBench, three exemplars use a `class Solution` functional scaffold and two use AtCoder-style stdin scaffolds, covering both observed `testtype` variants).
8. **Custom-Validated generation** (`31_creative_custom_generate.py`). Six hundred candidate creative prompts produced by Claude Sonnet (out-of-pool generator); SHA-256 of the artifact pinned in the lockfile.

9. **Custom-Validated filtering** (`32_creative_custom_validate.py`). Two-judge LLM pre-filter (GPT-4o + Gemini majority vote); ambiguous candidates routed to manual review; cap of 100 approved entries in v0.3.0.
10. **Assembly** (`40_assemble_eval_params.py`). Merges normalized rows, assigns deterministic `query_id`, fills the few-shot prefix into the `query` field via the per-dimension Jinja template (see Table 2), and emits `data/final/evaluation_parameters_full.jsonl`.
11. **Triple tokenization** (`50_tokenize_triplo.py`). Computes `input_tokens_qwen`, `input_tokens_deepseek`, `input_tokens_kimi` via `AutoTokenizer.from_pretrained(..., use_fast=True)`.
12. **Stratification report** (`60_stratify_report.py`). Emits `stratify.md` + `stratify.json`; enforces hard assertions on dimension counts, shots policy, and `query_id` uniqueness.
13. **Push** (`71_push_hub_multiconfig.py`). Direct file-level upload (`HfApi.upload_file`) of seven configs (all plus the six dimension subsets) to avoid in-memory materialization of large LiveCodeBench private-test blobs.

End-to-end runtime: approximately 60–90 minutes including all network operations and the synthetic few-shot generation step.

5 Few-shot Strategy

Few-shot composition follows three regimes (Table 2). For non-agentic, non-SimpleQA sources the chain-of-thought 5-shot template is used (see Listing 2 for the canonical Jinja rendering).

Table 2: Few-shot regime per source.

Regime	Sources	Shots
Native train pool	GSM8K, MMLU-Pro, LitBench, IFEval	5
Synthetic (qwen3.5-122b)	MATH-500, AIME-2025, LCB-v6, IFBench, GPQA, EQ-Bench	5
Zero-shot by design	SimpleQA (OpenAI protocol)	0
Zero-shot agentic	BFCL, tau-bench, GAIA, Planning-Custom, Custom-Valid.	0

Listing 2: Few-shot CoT template (`configs/prompts.yaml`).

```
Solved examples:

{% for ex in few_shot_examples %}
--- Example {{ loop.index }} ---
Question: {{ ex.question }}
Reasoning: {{ ex.reasoning }}
Final answer: {{ ex.final_answer }}

{% endfor %}
--- Question to solve ---
Question: {{ query }}
Reasoning:
```

For sources whose few-shot pool was synthetically generated, the corresponding pool file (`data/fewshot_pools/⟨source⟩.json`) is committed to the repository, its SHA-256 is recorded in the lockfile, and the generator model identifier (`qwen3.5-122b@regolo`) is recorded as provenance. This produces a stable, reproducible artifact even though the generator API is not deterministic across calls.

6 Tokenization

Pre-computed token counts under three tokenizers are persisted in-row to enable downstream cost analysis without re-tokenization. Tokenizer fidelity is asymmetric (Table 3): the Qwen3.5-9B tokenizer is the official tokenizer of the target Qwen model (exact); the DeepSeek-V3 and Kimi-K2.5 tokenizers serve as proxies for the closed-weight DeepSeek-V4-Flash and the Kimi-2.6 deployments, with an expected drift of $\pm 2\text{--}5\%$.

Table 3: Tokenizer assignment for the three target models.

Target model	Tokenizer (HF id)	Fidelity
qwen3.5-9b	Qwen/Qwen3.5-9B	exact
deepseek-v4-flash	deepseek-ai/DeepSeek-V3	proxy ($\pm 2\text{--}5\%$)
kimi2.6	moonshotai/Kimi-K2.5	proxy ($\pm 2\text{--}5\%$)

7 Gated Masking

Two upstream sources are subject to gated access (GAIA [11] and GPQA-Diamond [10]). To preserve license compliance for the public release, rows derived from gated sources are emitted with `query = <masked>`, `expected_answer.payload = <masked>`, and `gated = true`. A separate script (`99_unmask_gated.py`) restores the original payload for users with the relevant credentials, by re-fetching the upstream rows and joining on `query_id`.

8 Quality Validation

The dataset is validated across three tiers, each gating a strict acceptance threshold. Failure of any tier blocks the public push.

8.1 Tier 1 — Deterministic Checks

Thirteen deterministic checks (no API calls, executable in CI) are applied to every row of the assembled artifact. Table 4 reports the threshold and the observed value at `v0.3.0`.

Table 4: Tier-1 deterministic checks (`v0.3.0`).

Check	Tool	Threshold	<code>v0.3.0</code>
Schema (14 columns)	pydantic v2	100% pass	PASS
MinHash dedup ($\text{sim} \geq 0.75$)	datasketch	$< 2\%$	PASS
Embedding cosine ≥ 0.92	all-mpnet-base-v2 + FAISS	$< 1\%$	PASS
Tokenizer recompute	transformers fast	$< 0.1\%$ mismatch	PASS
UTF-8 NFC + <code>ftfy</code> mojibake	<code>ftfy</code>	0 occurrences	PASS
Payload typed validation	jsonschema (Draft 2020-12)	100%	PASS
13-gram contamination	rapidfuzz	$< 5\%$ per source	PASS
PII (extended)	Presidio (en+it)	0 critical	PASS
Toxicity (offline)	unitary/detoxify	0 score > 0.7	PASS
HF Hub round-trip	datasets	schema match	PASS

8.2 Tier 2 — LLM-as-Judge

Four judge tasks are executed by `qwen3.5-122b@regolo` with position-swap and majority-vote across three independent calls per item. The four tasks are: (i) few-shot coherence (5-axis

rubric: format, CoT validity, answer-CoT alignment, hallucination, sufficiency); (ii) answer sanity (judge re-derives `expected_answer` on a math/knowledge/coding subset and flags mismatches); (iii) capability tag verification (Bloom-style assignment check on a 10% stratified sample); (iv) routing-signal proxy (judge predicts per-model success and flags rows with unanimous prediction as low-discrimination). All four pass the configured thresholds at v0.3.0.

8.3 Tier 3 — Dual-Reviewer Manual Audit

A dual-reviewer human audit was conducted on a stratified sample of $n = 102$ queries. The sample was drawn proportional to source counts, with deliberate over-sampling of LiveCodeBench-v6 (20 rows), BFCL-v4 (20 rows), and SimpleQA (12 rows) to maximize statistical power on the historically defect-prone sources.

Reviewers. The two annotators were Massa, F. and Cristofanilli, M. Both reviewers worked independently from the same materialized HTML artifact, using identical 4-class rubric definitions distributed at the start of the audit:

- **ok** — well-formed query; few-shot exemplars are coherent with the target task; expected payload is non-empty (or empty by design as in BFCL-irrelevance category).
- **broken** — structural defect that makes evaluation impossible: empty expected payload when needed, placeholder text in few-shot answers, target prompt cut mid-word.
- **low_quality** — technically valid but few-shot exemplars mismatch target style (e.g. all functional examples for a stdin task), or templated/repetitive across rows.
- **ambiguous** — target output format unclear or multiple equally valid answers possible without explicit rubric.

Inter-annotator agreement. Cohen’s κ was computed both for the full 4-class rubric and for the binary collapse `{ok}` vs. `{broken, low_quality, ambiguous}`. Results are reported in Table 5.

Table 5: Inter-annotator agreement on the $n = 102$ stratified sample.

Metric	4-class	Binary (ok / not_ok)
Raw agreement	95.1% (97/102)	99.0% (101/102)
Cohen’s κ [1]	0.761	0.951
Threshold (substantial agreement)	≥ 0.70	≥ 0.70
Outcome	PASS	PASS
Defect rate (R1, Massa F.)	10.8% (11/102)	10.8%
Defect rate (R2, Cristofanilli M.)	11.8% (12/102)	11.8%

Disagreements. Five row-level disagreements were observed, distributed as follows: one disagreement on a LiveCodeBench-v6 row (R2 over-strict on a structurally well-formed functional task), and four disagreements on IFBench rows where the two reviewers concurred on the row being defective (binary agreement) but disagreed on the severity label (**broken** vs. **low_quality**, captured by the binary collapse). Both reviewers concurred on the two root causes: (i) LiveCodeBench-v6 `testtype` mismatch — stdin AtCoder-style targets paired with a functional-only synthetic few-shot pool; and (ii) IFBench placeholder-response artifact — the synthetic few-shot pool contained literal placeholder strings instead of constraint-satisfying exemplars.

Outcome. Both root causes were patched in the v0.3.0 revision: the LiveCodeBench-v6 few-shot pool was regenerated with three functional and two stdin AtCoder-style exemplars; the IFBench few-shot pool was regenerated with concrete constraint-satisfying responses. A subsequent $n = 30$ spot-check of the patched pools (Table 6) confirmed a residual defect rate consistent with a global estimate of approximately 2%, below the production-readiness threshold of 3%.

9 Iterative Remediation

Three revisions were produced over the course of construction. Table 6 reports the per-source defect rate observed at each revision and the patches applied between revisions.

Table 6: Per-source defect rate (Tier-3 manual audit) across revisions.

Source	v0.1.0	v0.2.0	v0.3.0	Patch applied
BFCL-v4	100%	0%	0%	ground_truth_calls populated via possible_answer/ join
SimpleQA	86%	0%	0%	explicit answer-format prefix added
Custom-Validated	100%	0%	0%	confirmed valid in v0.2 (R1-strictness in v0.1)
LiveCodeBench-v6	78%	40%	7%	few-shot pool regenerated with mixed scaffolds
IFBench	0%	100%	13%	few-shot pool regenerated with real responses
Globally estimated	45.6%	11.8%	~2.0%	

The IFBench regression at v0.2.0 (from 0% to 100%) was not detected in the v0.1.0 audit because the original $n=57$ stratified sample did not include any IFBench rows; the over-sampled v0.2.0 audit ($n=102$, four IFBench rows) surfaced the placeholder defect.

10 Evaluation Results

Beyond enabling routing-level decision evaluation, Dataset A doubles as a single-model capability benchmark. This section reports per-protocol results for all three target models (qwen3.5-9b, deepseek-v4-flash, kimi2.6) on all ten scoring protocols: five deterministically graded (gsm8k_final_answer, math_equiv, ifeval_constraint_check, lcb_unit_test, mcq_letter), two single-judge LLM-graded (llm_judge_factual, creative_rubric_judge), and three agentic protocols (BFCL single-turn tool_call_match, BFCL multi-turn, and planning rubric_judge). The agentic planning rubric is additionally evaluated under a three-judge panel (§10.4) to characterize judge-induced variance on the most subjective protocol.

10.1 Experimental Setup

The three models were served through different infrastructures. qwen3.5-9b (open-weight) was served locally on a four-way NVIDIA L40S node via vLLM (cu130 build, tensor-parallel size 4, max_model_len = 65,536, reasoning parser qwen3). deepseek-v4-flash (closed-weight) was accessed through the OpenRouter API. kimi2.6 (closed-weight) was also accessed via OpenRouter; in line with its longer reasoning traces, client-side concurrency was reduced from 16 to 8 (after an initial OOM on a 23 GiB host with concurrency 16), and on a skip-retry pass the per-protocol max_tokens caps were further raised (ifeval_constraint_check → 32,768, lcb_unit_test → 65,536, math_equiv → 49,152) to recover queries that had truncated empty on the initial pass (see §10.6). All three models were run with reasoning enabled, temperature = 0.0, top_p = 1.0, and a thinking-token budget of 16,384 where the runtime exposes the parameter; client-side request concurrency was 16 for qwen3.5-9b and deepseek-v4-flash.

A model-specific subtlety governs the per-protocol completion-token cap (`max_tokens`). Under vLLM with the `qwen3` reasoning parser the reasoning trace counts against the request’s `max_tokens`; if the cap is too small, the model can terminate with `finish_reason = length` before emitting a final answer. `deepseek-v4-flash` manages its reasoning budget internally and does not exhibit this behaviour. Caps were therefore sized for `qwen3.5-9b` (Table 7); `deepseek-v4-flash` ran at the same or smaller caps with a truncation rate $\leq 1\%$ on every protocol. `kimi2.6` turned out to share the `thinking-against-max_tokens` behaviour on the agentic subset (in contrast to the per-model assumption that originally placed it with `deepseek-v4-flash`): on a first agentic pass, 18 queries (1 `tool_call_match` +17 planning `rubric_judge`) saturated the cap entirely in reasoning, leaving zero output tokens. The cap was raised on a targeted retry pass (`tool_call_match` $\rightarrow$ 8,192, planning `rubric_judge` $\rightarrow$ 28,672, and a single residual query at 40,960); all 18 were recovered with `finish_reason = stop`.

Table 7: Completion-token caps (`max_tokens`) for `qwen3.5-9b`. The `rubric_judge` creative caps were raised from 24,576 to 40,960 for the subset of queries re-run after the truncation analysis of §10.6. The $\dagger$ values on the agentic rows reflect the `kimi2.6` retry pass described in the previous paragraph.

Protocol	Cap
<code>gsm8k_final_answer</code>	32,768
<code>math_equiv</code>	32,768
<code>ifeval_constraint_check</code>	32,768
<code>lcb_unit_test</code>	49,152
<code>mcq_letter</code>	24,576
<code>llm_judge_factual</code>	24,576
<code>rubric_judge</code> (creative)	24,576 / 40,960
<code>rubric_judge</code> (planning)	20,480 / 28,672 / 40,960 †
<code>tool_call_match</code>	2,048–4,096 / 8,192 †

10.2 Grading Methodology

Eight of the ten evaluation protocols are graded deterministically, with no API calls: `gsm8k_final_answer` (exact match on the normalized final answer), `math_equiv` (SymPy symbolic-equivalence judging), `ifeval_constraint_check` (the official IFEval/IFBench programmatic constraint checkers), `lcb_unit_test` (sandboxed execution against the LiveCodeBench public and private test suites, per-test timeout 6s), `mcq_letter` (single-letter extraction), and `tool_call_match` (BFCL abstract-syntax-tree matching for single-turn calls; the BFCL v4 multi-turn state-based checker for the multi-turn subset, where the model holds an inner-loop dialogue with the simulated tool environment and final correctness is judged by deterministic comparison between the model’s post-trajectory state and the ground-truth state).

The remaining two protocols use an LLM-as-judge. A single judge model, `openai/gpt-5.4-mini` accessed via OpenRouter, grades both `rubric_judge` (creative writing and agentic planning) and `llm_judge_factual` (SimpleQA-style short-form factuality). The judge is cross-provider relative to the evaluated pool, avoiding self-preference bias, and is run at `temperature = 0.0`, `top_p = 1.0`. For `llm_judge_factual` the canonical OpenAI SimpleQA grader template is used, yielding a three-class label—A (CORRECT), B (INCORRECT), C (NOT_ATTEMPTED)—which map to `correct` $\in$ {True, False, None} respectively. For `rubric_judge` the judge applies a fixed criterion-separated rubric and emits a final `Decision: accept|reject` line (`accept` $\rightarrow$ True). Every judge call’s raw output is logged verbatim for post-hoc audit; `temperature = 0.0` on the upstream mixture-of-experts backend is best-effort, not bit-reproducible. The single-judge LLM-judge grading of the final artifacts cost \$4.53 in total.

Three-judge panel for agentic planning. Agentic planning is the most subjective of the protocols: the rubric is *conjunctive* (all applicable criteria must pass) and admits substantial judge-dependent variance. To characterize this we additionally grade the 335 `rubric_judge` (planning) queries under a three-judge panel: `openai/gpt-5.4-mini` (the canonical strict judge), `mistralai/mistral-small-2603`, and `z-ai/glm-5-turbo`, all cross-provider relative to the evaluated pool *and* to each other (three distinct model families: OpenAI, Mistral, Zhipu). Each judge runs at `temperature` = 0.0 with the identical rubric prompt. Per-query verdicts are aggregated by majority vote on the two-of-three valid (non-abstention) verdicts; ties and fewer than two valid votes resolve to `correct` = None. Panel results are reported in §10.4; the panel grading of the final artifacts cost \$7.30 across all three models.

Metrics. For each (model, protocol) pair we report three quantities. The *completion rate* is the fraction of queries that produced a gradable answer, $(C+I)/N$. The *conditional accuracy* is $C/(C+I)$, the accuracy over gradable answers. The *overall accuracy* is C/N , which charges ungradable answers as failures. For `llm_judge_factual` the NOT_ATTEMPTED class is treated as in the SimpleQA protocol: a model abstention rather than a harness error, excluded from the conditional-accuracy denominator but still counted against overall accuracy. All figures are regenerated from the graded artifacts by a committed aggregation script (`scripts/130_aggregate_results.py`), consistent with the staged, reproducible pipeline of §4.

10.3 Results

Table 8 reports the full per-protocol breakdown.

Across the five deterministic protocols, `kimi2.6` leads on four and ties on `gsm8k_final_answer` (all three models within 0.2 pp, near-saturated at $\geq 96.4\%$). The widest deterministic gap is on competitive coding (`lcb_unit_test`: 90.5% for `kimi2.6` vs. 82.1% for `deepseek-v4-flash` vs. 71.4% for `qwen3.5-9b`). On `math_equiv`, `kimi2.6` leads `deepseek-v4-flash` by 1.7 pp (92.6% vs. 90.9%); on `ifeval_constraint_check` the overall-accuracy ordering is `kimi2.6` 87.0% > `deepseek-v4-flash` 86.3% > `qwen3.5-9b` 81.0%; on `mcq_letter`, `kimi2.6` 73.5% > `deepseek-v4-flash` 72.5% > `qwen3.5-9b` 64.7%.

On the two judge-graded protocols (`llm_judge_factual` and creative `rubric_judge`), all three models are evaluated and `kimi2.6` leads both on conditional accuracy: 59.6% vs. 49.8% vs. 11.9% on factuality and 75.7% vs. 65.8% vs. 51.0% on creative writing. However, on `llm_judge_factual` `kimi2.6` only attempts 48.1% of the queries (337/700 answered, 363/700 NOT_ATTEMPTED), so its overall accuracy is 28.7% – below `deepseek-v4-flash`’s 45.6%. This calibration trade-off is analysed in §10.6. On the three agentic protocols all three models are evaluated. `kimi2.6` leads on all three: single-judge planning `rubric_judge` (14.6% vs. `deepseek-v4-flash` 13.7% vs. `qwen3.5-9b` 5.4%); BFCL single-turn `tool_call_match` (94.4% vs. 93.0% vs. 93.0%, near-saturated); BFCL multi-turn (21.2% vs. 18.2% vs. 20.6%, all weak). The planning rubric figures are the strictest available reading under a single canonical judge; the three-judge panel reading – which yields the same ranking but markedly higher absolute accuracies – is reported in §10.4.

10.4 Agentic planning under a three-judge panel

The planning `rubric_judge` accuracies in Table 8 sit between 5.4% and 14.6% – an order of magnitude below the other `rubric_judge` subset (creative writing, 51–76%) for the same models. Manual inspection of the rejected planning queries confirms that `openai/gpt-5.4-mini` rejects for substantive reasons: required tools left unused, explicit constraints (budget, deadline) violated, or hypothetical and non-actionable steps substituted for concrete ones. The protocol is genuinely hard at this model scale; the rubric is genuinely demanding. Nonetheless, single-judge LLM grading on a conjunctive subjective rubric is known to be sensitive to the judge’s threshold, and

Table 8: Per-protocol evaluation results (v0.3.0, $N=5,339$). C/I = correct/incorrect; NA/tr = not-attempted (llm_judge_factual) or truncated/errored. Compl. = completion rate; Acc_c = conditional accuracy; Acc_o = overall accuracy.

Protocol	Model	N	C	I	NA/tr	Compl.	Acc_c	Acc_o
gsm8k_final_answer	qwen3.5-9b	470	454	16	0	100.0%	96.6%	96.6%
	deepseek-v4-flash	470	453	17	0	100.0%	96.4%	96.4%
	kimi2.6	470	453	17	0	100.0%	96.4%	96.4%
math_equiv	qwen3.5-9b	530	458	50	22	95.9%	90.2%	86.4%
	deepseek-v4-flash	530	482	48	0	100.0%	90.9%	90.9%
	kimi2.6	530	491	39	0	100.0%	92.6%	92.6%
ifeval_constraint_check	qwen3.5-9b	841	681	160	0	100.0%	81.0%	81.0%
	deepseek-v4-flash	841	726	115	0	100.0%	86.3%	86.3%
	kimi2.6	841	732	87	22	97.4%	89.4%	87.0%
lcb_unit_test	qwen3.5-9b	1,000	714	286	0	100.0%	71.4%	71.4%
	deepseek-v4-flash	1,000	821	178	1	99.9%	82.2%	82.1%
	kimi2.6	1,000	905	71	24	97.6%	92.7%	90.5%
mcq_letter	qwen3.5-9b	102	66	36	0	100.0%	64.7%	64.7%
	deepseek-v4-flash	102	74	28	0	100.0%	72.5%	72.5%
	kimi2.6	102	75	27	0	100.0%	73.5%	73.5%
llm_judge_factual	qwen3.5-9b	700	77	571	52	92.6%	11.9%	11.0%
	deepseek-v4-flash	700	319	322	59	91.6%	49.8%	45.6%
	kimi2.6	700	201	136	363	48.1%	59.6%	28.7%
rubric_judge (creative)	qwen3.5-9b	696	355	341	0	100.0%	51.0%	51.0%
	deepseek-v4-flash	696	458	238	0	100.0%	65.8%	65.8%
	kimi2.6	696	524	168	4	99.4%	75.7%	75.3%
rubric_judge (planning)	qwen3.5-9b	335	18	317	0	100.0%	5.4%	5.4%
	deepseek-v4-flash	335	46	289	0	100.0%	13.7%	13.7%
	kimi2.6	335	49	286	0	100.0%	14.6%	14.6%
tool_call_match (BFCL)	qwen3.5-9b	500	465	34	1	99.8%	93.2%	93.0%
	deepseek-v4-flash	500	465	35	0	100.0%	93.0%	93.0%
	kimi2.6	500	472	28	0	100.0%	94.4%	94.4%
tool_call_match (BFCL multi-turn)	qwen3.5-9b	165	34	130	1	99.4%	20.7%	20.6%
	deepseek-v4-flash	165	30	134	1	99.4%	18.3%	18.2%
	kimi2.6	165	35	119	11	93.3%	22.7%	21.2%

the gap between 5.4% and 14.6% is small enough in absolute terms that one might reasonably ask how much of it reflects model capability versus judge stance. We address this by re-grading the same 335 planning queries under a panel of three cross-provider judges and aggregating by majority vote.

Table 9 reports the per-judge conditional accuracy and the panel verdict. The three judges, run at `temperature` = 0.0 on the identical rubric prompt, disagree dramatically in absolute terms. `openai/gpt-5.4-mini` accepts only 5–15% of planning queries; `mistralai/mistral-small-2603` accepts 90–98% of the same queries; `z-ai/glm-5-turbo` sits between, at 52–72%. The inter-judge spread is largest on `kimi2.6` (83.6 pp between the most lenient and most strict judge). The panel verdict, by construction the median of the three for a 3-judge majority, tracks GLM almost exactly (within ≤ 0.5 pp on every model) – as expected when the strict and lenient extremes outvote each other and the median voter decides.

The vote distribution underneath these aggregates further qualifies the result. Counting accept votes among the three judges per query, the distribution is concentrated on the split

Table 9: Three-judge panel results on `rubric_judge` (planning, $N=335$). Acc_c is conditional accuracy ($T/(T + F)$); the **panel-2of3** row aggregates by majority vote on the valid (non-abstention) verdicts and resolves ties / < 2 valid votes to **None**. Abst. = queries on which the panel could not reach a 2-of-3 majority (judge output not parseable into accept/reject).

Model	Judge	T	F	Abst.	Acc_c
qwen3.5-9b	openai/gpt-5.4-mini	18	317	0	5.4%
	mistralai/mistral-small-2603	301	34	0	89.8%
	z-ai/glm-5-turbo	173	162	0	51.6%
	panel-2of3	173	162	0	51.6%
deepseek-v4-flash	openai/gpt-5.4-mini	46	289	0	13.7%
	mistralai/mistral-small-2603	320	15	0	95.5%
	z-ai/glm-5-turbo	225	108	2	67.6%
	panel-2of3	228	105	2	68.5%
kimi2.6	openai/gpt-5.4-mini	49	286	0	14.6%
	mistralai/mistral-small-2603	329	6	0	98.2%
	z-ai/glm-5-turbo	240	93	2	72.1%
	panel-2of3	241	92	2	72.4%

outcomes: **qwen3.5-9b** produces 156 2–1 accepts and 129 1–2 rejects, with only 17 unanimous accepts and 33 unanimous rejects; **deepseek-v4-flash** produces 185 2–1 and 90 1–2, with 43 unanimous accepts and 15 unanimous rejects; **kimi2.6** produces 193 2–1 and 86 1–2, with 48 unanimous accepts and only 6 unanimous rejects. The monotonic shift in the unanimous-accept and unanimous-reject counts across the three models (Qwen 17/33 $\rightarrow$ DeepSeek 43/15 $\rightarrow$ Kimi 48/6) mirrors the panel ranking and is reassuringly robust to the choice of any single judge.

Reading the gap. The single-judge canonical numbers (Table 8, 5.4–14.6%) and the panel numbers (Table 9, 51.6–72.4%) are not in tension: the former is the strictest of the three readings (the canonical judge has the highest threshold), the latter is the median. They are both valid summaries of the same set of 1,005 accept/reject decisions ($3 \text{ judges} \times 335 \text{ queries}$), differing only in how those decisions are aggregated. What matters scientifically is that the *ranking* **qwen3.5-9b** < **deepseek-v4-flash** < **kimi2.6** is preserved on every judge and on the panel, so the comparative conclusion is robust even though the absolute accuracy on this protocol is judge-dependent. We therefore report both readings: the single-judge numbers for cross-protocol comparability with the rest of the table, the panel numbers for a less threshold-sensitive view of agentic planning capability.

Caveats. The pairwise Cohen’s κ between **openai/gpt-5.4-mini** and **mistralai/mistral-small-2603** on this protocol is close to zero, reflecting the wide threshold gap and not a random-guessing baseline – the two judges agree on which queries are *best* (the model ranking is preserved) but disagree on where to set the accept/reject threshold. The panel-as-median heuristic mitigates this mechanically but does not anchor the threshold to any external ground truth; a human-anchored $\kappa(\text{judge}, \text{human})$ over a stratified 50–100-query subset is the recommended next step.

10.5 Aggregate Per-Query Accuracy and Oracle Bound

While Table 8 reports per-protocol accuracy, the routing literature typically compares models on aggregate per-query response accuracy and against the achievable upper bound when an oracle selects the optimal model per query. This subsection reports those metrics on the full Dataset A corpus ($N=5,504$, including the 165 **planning_agentic_multiturn** queries added in v0.4 that augment the original v0.3.0 release of 5,339 rows).

Table 10: Standalone per-model response accuracy on Dataset A ($N=5,504$). A query is *solved* iff the model’s response passes the per-protocol grader. No routing is involved.

Model	Solved	Response accuracy
qwen3.5-9b	3,477 / 5,504	63.17%
deepseek-v4-flash	4,056 / 5,504	73.69%
kimi2.6	4,129 / 5,504	75.02%

Table 11: Oracle upper bound: maximum achievable response accuracy when an oracle selects the optimal model per query, for progressively larger model pools.

Pool	Oracle solved	Oracle accuracy
kimi2.6 standalone (no routing)	4,129 / 5,504	75.02%
kimi2.6 + deepseek-v4-flash	4,511 / 5,504	81.96%
qwen3.5-9b + ds4 + kimi2.6 (full)	4,582 / 5,504	83.25%
Unsolvable by any pool model	922 / 5,504	16.75%

Despite kimi2.6 leading on most protocols (Table 8), it is not the per-query oracle: it fails 1,375 queries, of which 453 are correctly solved by at least one of the cheaper models in the pool (382 by `deepseek-v4-flash` and 71 by `qwen3.5-9b` alone). Consequently, an oracle restricted to (kimi2.6, `deepseek-v4-flash`) reaches 81.96% response accuracy – a 6.94 percentage-point improvement over kimi2.6 standalone – and the full three-model oracle reaches 83.25% (Table 11).

The Venn breakdown (Table 12) bounds the achievable routing gain. The 382 queries where only `deepseek-v4-flash` succeeds (5.23% of the corpus) plus the 94 where `qwen3.5-9b` and `deepseek-v4-flash` both succeed but kimi2.6 fails (1.71%) constitute the principal routing upside – capturing them moves an intelligent router strictly above any single-model baseline. The 922 unsolvable queries (16.75%) define the irreducible ceiling: closing this gap requires extending the pool with a stronger fourth model rather than smarter routing within the current three.

10.6 Discussion

Reasoning-budget truncation and degenerate generation. An initial `qwen3.5-9b` run exhibited high `finish_reason = length` rates on `ifeval_constraint_check` (63%), `mcq_letter` (27%), `lcb_unit_test` (20%) and creative `rubric_judge` (20%), because the per-protocol `max_tokens` caps were smaller than the thinking budget plus the answer, leaving an empty (ungradable) response. Caps were raised (Table 7) and the affected queries re-run. Re-running resolved the empty-answer cases but exposed a second, distinct failure mode: on 6.7% of `ifeval`, 8.2% of `lcb` and 9.5% of creative `rubric_judge` queries the model enters a *degenerate generation loop*—emitting tens of thousands of characters of repeated text (e.g. a code comment or a sentence repeated until the token cap). These are not recoverable by a larger budget; they are genuine model failures and are graded as incorrect. They account for a substantial part of `qwen3.5-9b`’s deficit on the coding, instruction-following and creative protocols, and constitute a robustness finding for 9B-class reasoning models. `deepseek-v4-flash` did not exhibit either failure mode at a material rate ($\leq 1\%$ truncation throughout). The only residual ungradable cases are 22 `math_equiv` queries for `qwen3.5-9b` (4.2%, below the re-run threshold).

Kimi2.6 unbounded-thinking truncation. `kimi2.6` exhibits a third truncation pattern, distinct from `qwen3.5-9b`’s empty-answer and degenerate-loop modes: a small fraction of queries cause its reasoning trace to exceed the per-call `max_tokens` budget on both the initial pass and a retry pass with raised caps. The final residual ungradable rate on the deterministic protocols is `lcb_unit_test` 2.4%, `ifeval_constraint_check` 2.6%, and 0% on `math_equiv`, `mcq_letter`,

Table 12: Per-query solver overlap (Venn) across the three-model pool ($N=5,504$). Each row reports the (qwen3.5-9b, deepseek-v4-flash, kimi2.6) solver status as (T, F). Bold rows highlight the routing opportunities where a cheaper or middle-tier model rescues a kimi2.6 failure.

qwen	ds4	kimi	N	%	Notes
T	T	T	3,106	56.43%	easy: all three solve
F	T	T	568	10.32%	ds4+kimi (qwen fail)
F	F	T	249	4.52%	only kimi rescues
F	T	F	288	5.23%	only ds4 rescues (kimi fails!)
T	F	T	206	3.74%	qwen+kimi (ds4 fail)
T	T	F	94	1.71%	qwen+ds4 (kimi fails!)
T	F	F	71	1.29%	only qwen rescues
F	F	F	922	16.75%	unsolvable (irreducible)
Total			5,504	100.00%	

and `gsm8k_final_answer`. On the retry pass we attempted an explicit reasoning-budget cap via OpenRouter’s `reasoning.max_tokens` field (a 20,000-token cap applied to the reasoning-only portion): on a 12-row pilot, the cap was honoured on only 7/12 queries; on the remaining 5/12 `kimi2.6` emitted between 25 *k* and 41 *k* reasoning tokens, indicating the parameter is not enforced by this provider. We therefore fell back to raising the joint `max_tokens` cap alone (`lcb_unit_test` $\rightarrow$ 65,536, `math_equiv` $\rightarrow$ 49,152, `ifeval_constraint_check` $\rightarrow$ 32,768), which recovered 142/188 of the initial skips. The residual 46 rows have no in-protocol mechanism to bound the trace and are graded as None (skip-on-truncation), contributing to the Acc_c versus Acc_o gap on `ifeval_constraint_check` and `lcb_unit_test`.

Agentic planning is the hard frontier. Across the three agentic protocols, the picture is one of sharp asymmetry. Single-turn `tool_call_match` is near-saturated for all three models (93–94%), indicating that the BFCL single-turn function-calling format is no longer discriminative at this model scale. The two harder agentic protocols collapse in the opposite direction: BFCL multi-turn lands in the 18–21% band for all three models, and planning `rubric_judge` lands in the 5.4–14.6% band under the canonical single judge – or, more threshold-robustly, in the 51.6–72.4% band under the three-judge panel (Table 9). Manual inspection of the planning rejections under the canonical judge confirms substantive failure modes: required tools left unused, explicit constraints (budget, deadline) violated, or hypothetical and non-actionable steps substituted for concrete ones. The conjunctive rubric (all applicable criteria must pass) compounds the difficulty. Critically, the panel-versus-canonical gap on planning does not reorder the models: on every judge and on the panel, `kimi2.6` $>$ `deepseek-v4-flash` $>$ `qwen3.5-9b`. The multi-turn results paint a similar weakness story across providers ($\leq 22.7\%$ conditional accuracy on a deterministic state-based checker) and suggest that multi-step agentic tool-use, not function-calling format, is the bottleneck at this scale. A graded per-criterion planning score and an explicit decomposition of multi-turn failures into tool-selection vs. state-tracking errors are left to future work.

Short-form factuality. On SimpleQA-style `llm_judge_factual`, `qwen3.5-9b` is correct on only 11.9% of attempted queries, against 49.8% for `deepseek-v4-flash` and 59.6% for `kimi2.6`. Abstention behaviour, however, differs sharply across the three models: `qwen3.5-9b` and `deepseek-v4-flash` decline only ≈ 7 –8% of queries, so their conditional/overall accuracies are close, whereas `kimi2.6` declines 51.4% of queries – collapsing its overall accuracy to 28.7% despite the higher conditional rate. The `qwen3.5-9b` vs. `deepseek-v4-flash` gap is consistent with parametric-knowledge capacity scaling with model size; the `kimi2.6` pattern is qualitatively distinct and is analysed next.

Calibration and hallucination rates. The three models exhibit qualitatively different calibration behaviours on the two judge-graded protocols. Defining the *hallucination rate* $H = F/(T + F)$ on attempted queries only (i.e. excluding NOT_ATTEMPTED abstentions, following the SimpleQA convention [9]), we observe:

Protocol	qwen3.5-9b	deepseek-v4-flash	kimi2.6
llm_judge_factual	88.1%	50.2%	40.4%
rubric_judge (creative)	49.0%	34.2%	24.3%

kimi2.6 shows the lowest hallucination rate on both protocols, but the pattern is most pronounced on llm_judge_factual, where it declines 51.4% of queries (NOT_ATTEMPTED) versus 7–8% for the other two models. Inspection of the 360 declined responses shows 337 are the literal string “*I don’t know*”, preceded on average by 1,644 reasoning tokens (median 1,316) – i.e. an active refusal after substantial deliberation, not a parse artefact. On the very same 360 queries qwen3.5-9b answers incorrectly 81.4% of the time and deepseek-v4-flash 53.3%, indicating the declined items are genuinely difficult and that kimi2.6’s abstention is selective rather than blanket. This trade-off – lower coverage in exchange for higher reliability when answering – is precisely the behaviour the SimpleQA framework [9] is designed to reward, and it suggests the model has been calibration-trained to prefer honest abstention over fabrication.

Scope. All three target models are now evaluated on all ten protocols, including the three agentic protocols (planning rubric_judge under both the canonical single judge and the three-judge panel, single-turn tool_call_match, and multi-turn tool_call_match). The three-judge panel is currently applied only to the planning rubric; extending it to the creative rubric_judge and to llm_judge_factual is methodologically straightforward and left to follow-up. A human-anchored $\kappa(\text{judge}, \text{human})$ on a stratified 50–100-query subset of the planning rubric is the recommended next step to validate the panel-as-median heuristic. The routing-system evaluation is the subsequent step.

11 Limitations

We document the following limitations.

1. **Tokenizer proxies.** Two of the three tokenizers (DeepSeek-V3 for DeepSeek-V4-Flash; Kimi-K2.5 for Kimi-2.6) are proxies for closed-weight target models. Drift relative to the true production tokenizer is estimated at $\pm 2\text{--}5\%$ and is documented in the dataset card. Cost-tier estimation downstream should treat these counts as lower-bound proxies.
2. **Gated masking.** GAIA ([11]) and GPQA-Diamond ([10]) rows are masked in the public release. Users with credentialed access can restore the original payloads via `99_unmask_gated.py`; routing evaluations on these subsets must be performed locally on the unmasked variant.
3. **Custom-Validated genre clustering.** The 100 approved Custom-Validated entries exhibit a residual genre clustering toward poetic / lighthouse / ocean themes (observed in approximately 30% of the pool). Future expansion to the full target of 404 entries should oversample under-represented genre tags.
4. **AIME 2024 dropped.** AIME 2024 was excluded due to insufficient statistical power at $n=10$ post-cutoff prompts and unresolvable contamination concerns. Only AIME 2025 ($n=30$) is retained.
5. **Multi-IF excluded.** The Multi-IF benchmark (cc-by-nc-2.0) was excluded because its license is incompatible with the public push. The 200 instruction-following slots vacated by Multi-IF were reallocated to additional IFEval rows (apache-2.0).

6. **Synthetic few-shot pools.** For the six sources without a native train split, the few-shot exemplars are synthetic (generated by `qwen3.5-122b@regolo`). Routing evaluations consume this pool consistently across all three target models, so the synthetic pool affects absolute scores but not relative ranking — which is the routing-evaluation primitive of interest.
7. **Irreducible pool ceiling.** As reported in §10.5 and Table 11, 922 queries (16.75% of the corpus) are not solved by any of the three target models, which bounds the achievable response accuracy of *any* router restricted to this pool at 83.25%. Closing the gap requires extending the pool with at least one model strictly stronger on these queries (e.g., a larger frontier model), not refining the routing logic itself.

12 Distribution and Reproducibility

Dataset A is publicly distributed on the Hugging Face Hub under `massaindustries/dataset-A-routing-eval` (revision 73aa0a09). The repository is laid out in the multi-config format with seven configurations: a top-level `all` configuration ($N=5,339$) plus six per-dimension configurations matching the row counts in Table 1. The top-level license is `other`; per-source license is recorded in-row in the `license` field.

Reproducibility is supported by (i) per-source upstream revision SHAs pinned in `data/reports/lockfile.yaml`; (ii) SHA-256 of every synthetic few-shot pool committed alongside the generator-model identifier; (iii) deterministic seeds in every sampling and shuffling stage; (iv) a final sort by `query_id` after all shuffles to remove cross-revision instability; and (v) a smoke-test script (`80_verify_dataset.py`) that re-loads the published artifact, re-tokenizes a 50-row sample, and asserts schema and token-count invariants.

References

- [1] J. Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.
- [2] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. *NeurIPS Datasets and Benchmarks Track*, 2021.
- [3] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [4] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.
- [5] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.
- [6] V. Pyatkin et al. IFBench: An instruction-following benchmark for LLM robustness evaluation. *AllenAI Technical Report*, 2025.
- [7] F. Yan, H. Mao, C. C.-J. Ji, T. Zhang, S. G. Patil, I. Stoica, and J. E. Gonzalez. Berkeley function calling leaderboard. <https://gorilla.cs.berkeley.edu/leaderboard.html>, 2024.

- [8] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- [9] J. Wei, N. Karina, H. W. Chung, Y. J. Jiao, S. Papay, A. Glaese, J. Schulman, and W. Fedus. SimpleQA: Measuring short-form factuality in large language models. *OpenAI Technical Report*, 2024.
- [10] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark. *COLM*, 2024.
- [11] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom. GAIA: A benchmark for general AI assistants. *arXiv preprint arXiv:2311.12983*, 2023.
- [12] S. J. Paech. EQ-Bench: An emotional intelligence benchmark for large language models. *arXiv preprint arXiv:2312.06281*, 2024.
- [13] Stanford CRFM. LitBench: A benchmark for long-form literary writing evaluation. <https://huggingface.co/datasets/SAA-Lab/LitBench-Test>, 2025.
- [14] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- [15] L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- [16] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.